

database and returning them as objects or groups of objects. One of the simpler queries is `.objects.first()`, which returns the first item in the database of that object type. I don't find I use this method much in my views, but it can be handy if you're mucking around in the REPL and need a quick piece of data to play with.

To fetch Mr. Vai's `Musician` object, you'll first need to re-import `Musician`, as closing the shell before will have lost the import state. Once you have the `Musician` class, you run the `.first()` method on the `.objects` query manager, and it returns the first (and only) `Musician` object:

```
(venv) RiffMates$ python manage.py shell
Python 3.12.1
Type "help", "copyright", "credits" or "license" for more information.
(InteractiveConsole)
>>> from bands.models import Musician
>>> steve = Musician.objects.first()
>>> steve
<Musician: Musician(id=1, last_name=Vai)>
```

With your new `__str__()` in place, the display value of `steve` in the REPL is more readable. Steve is kind of lonely; use `.create()` to add a few more musicians to your database:

```
>>> from datetime import date
>>> Musician.objects.create(first_name="John", last_name="Lennon",
➤ birth=date(1940, 10, 9))
<Musician: Musician(id=2, last_name=Lennon)>
>>> Musician.objects.create(first_name="John", last_name="Bonham",
➤ birth=date(1948, 7, 31))
<Musician: Musician(id=3, last_name=Bonham)>
```

You have some data to play with and a fantasy rock trio that would make any record company salivate. As you might guess, to go along `.first()`, you'll need a `.last()` query, which returns the most recent item added to the database. Try it out on your own. Fetching a single object is kind of boring though, so you can get sets of data and filter the content as well.

4.4.1 Using `.all()` and `.filter()` to fetch `QuerySet` results

To find all the objects in a table, you call the query manager's `.all()` method. This returns a `QuerySet` object, a wrapper to the query you ran. Try it out yourself:

```
>>> Musician.objects.all()
<QuerySet [<Musician: Musician(id=1, last_name=Vai)>, <Musician: Musician(id=2, last_name=Lennon)>, <Musician: Musician(id=3, last_name=Bonham)>]>
```

Individual items, or a subset of the result of a `QuerySet`, can be accessed using square brackets, similar to a list or tuple. Re-run the `.all()` query, store the result, and then access the second item using `[1]`:

```
>>> result = Musician.objects.all()
>>> result[1]
<Musician: Musician(id=2, last_name=Lennon)>
```